Homework 7

Atharva Chalke

May 24, 2018

Problem 1. A student proposed a different design for the enumerator we discussed when proving that any NDTM can be simulated by a DTM. The design, for b= 3, is shown at: https://piazza.com/class/jg875z28of53z0?cid=48Remember that the enumerator must create all the strings with k digits before it creates any with k+ 1 digits. But within the group of strings with the same number of digits, the order of the strings does not matter.Does this design work as an enumerator for b= 3? Explain. Explain how the design would need to be changed for an arbitrary value of b.

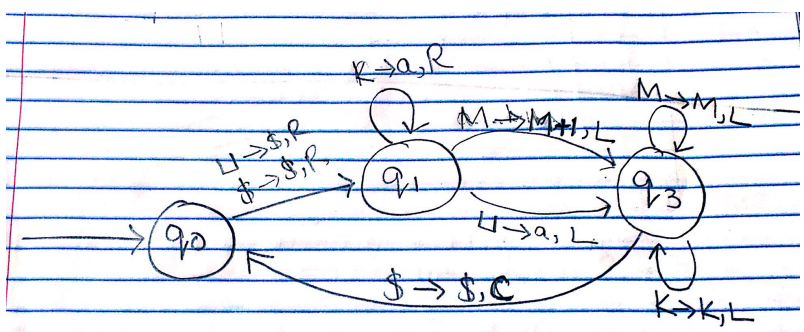
Proof. This will not work, since the initial condition is on seeing a dollar sign, add a dollar sign and move right. But initially there is not dollar sign so the machine will not proceed.
A modification is on seeing a blank space or a dollar sign , we add a $to denote left end.
For b=3
This will work. The string outputted will not be ordered, but all enumerations will be outputted before moving on to the next digit.
Ex:
$1
$2
$3
$11
$21
$31
$12
$22
$32
$13
$23
$33
$12
$22
$111
...

Change for any arbitrary b

The basic outline of the algorithm will work, but the following changes will need to be made for an arbitrary b.

The following diagram shows the changes made.

Lets say b = k.
Let the lexicographic minimum of the k values available be a.
(For example: If we have {1,2,3} then minimum of the values is 1.)
Let the lexicographic maximum of the k values available be K.
(For example: If we have {1,2,3} then maximum of the values is 3.)
Let M denote all values other than K.
(For example: If we have {1,2,3} then M = {{1},{2}}.)
(M+1) is the next character when all characters are arranged lexicographically.



Scanned with CamScanner

□

Problem 2. Sipser 3.14

Proof. Proving that a DTM is equivalent to a Queue Automaton is relatively easy. Since, for a Queue automaton we can write on left end, and read on right end, but for a DTM we the read-write head is flexible, so it is really easy to simulate a Queue Automaton.

Simulating a Queue Automaton on a DTM
In class, we have established that multiple tapes are equivalent to a single tape, so lets use this here.
We have 2 tapes.
Lets call tape 1 INPUT TAPE
Lets call tape 2 QUEUE TAPE
We know multiple read-write heads are equivalent to a single read-write head, as proved in class. So lets use this here.
INPUT OPERATIONS
For the input head, we can mark the character we are reading on the input string, and we are only allowed to move right of the character being read on this tape. Thus, we simulate the QA since it is only allowed to move right on the input tape.
For initializing this process, we add a $symbol at the left end of the input tape to detect left end.
As the Queue automaton has spaces after its input to detect end of input, so does a DTM. Therefore, the input simulation is done.
QUEUE OPERATIONS
For the Queue Tape, we need to push on left end, and pull at the right end.
Lets use a $symbol to denote left end here.
For every push operation, we shift everything to the right of the dollar symbol one space to the right on the queue tape, and then add the new character to the right of the dollar symbol.
For every pull operation, we go to the right end, and pop the character there. The right is easy to mark, since we move right till we see a space, and the character to the left of it is the last character in the Queue.
Thus, the Queue automaton can be simulated.

Simulating a DTM on Queue Automaton
A formal description of a TM includes the following:
1. Q - Finite set of states
2. Γ - Finite non empty set of tape alphabet.
3. q_0- initial state
4. Final state - Final state
5. δ : Q X $\Gamma \rightarrow$ Q X Γ X (L,R)
6. $\Sigma \in \Gamma$

We need to represent all of this in a QA.
We can use binary strings to denote everything, this can be programmed into the Finite state Machine of the QA.

Thus, the alphabet will only need 1s and 0s.
Thus, the states and input alphabet is taken care of.

Step 1
The Finite state controller of the TM is the same as Finite state controller of the QA.

Step 2
Copy all of the contents of the TM tape to the tape of QA.
(EX: IF TAPE CONTAINS (a,b,c,d,e) copy (a,b,c,d,e)

Use a marker such as $at the left end to mark beginning, and # to mark right end of this content. (Ex: ($,a,b,c,d,e,#)

We also use a symbol to mark the read-write head, lets call it H.
This is placed to the right of the $symbol(left end) initially.(DO NOT REPLACE THE CONTENT of the tape JUST ADD A SPACE THERE AND ADD A H SYMBOL THERE)
Ex: ($,H,a,b,c,d,e,#)

WE ASSUME THAT H IS READING THE CHARACTER TO THE RIGHT OF IT
For ex:
($,H,a,b,c,d,e,#) In this we are reading a.

Simulating a right move on QA
We add a two state buffer to our FSM.

Description of a two state buffer:
1. It takes in a character at a time, and stores it on its queue.
2. If initially the character is a # and the next character we see is a H. Then, we output # and then H (In this case, we either have no input or we have reached the right most end).
3. If the previous case is False then move on.
4. Then after the # , we will have at least one character other than H, and #.
5. Write this on the buffers queue stack and output the #.
6. Check if the next character is a H or a character other than H.
7. If it is H, then empty queue for the buffer. Then push H first, and then Push the character we want to write. Output everything on buffer queue.
8. If the next character is not H, then output the character on the queue, and write the next character onto the queue. 9. Go to step 6.
10. After every iteration check if there is a # after the H symbol. If there is then, shift H one place left.

We consult our two state buffer for a right move.
So lets say in the example above we are reading "a" and then we want to replace a with "k" and then move right.
In the 2 state buffer, we see a character, in this case it's "a", and then we see "H". We push

in "H" first followed by the replacement character "k".
Therefore, our tape is now - ($,k,H,b,c,d,e,#) and we are reading b.

Simulating a left move on QA
For simulating a left move, we need a three state buffer. To remember the character before H, after H, and H.

Description of three state buffer:
1. It takes in a character at a time, and stores it on its queue.
2. If initially the character is a # and the next character we see is a H, and then we see a $. Then, we output # and then H, and then $. (In this case, we either have no input or we have reached the left most end).
3.If the previous case is False, then move on.
4. Then after the #, we will have atleast one character other than H and # and $.
5. Write this on the buffer stack, and output #.
6.Check if the next character is H or not H.
7. If next character is not H, output the character on stack, and push the new character onto the stack.
8. If the next character is H, do the following.
9. Push H onto the stack.
10. Push the character after H onto the stack.
11. If the character after H is a $. Then remove the first character on rightmost end on the Queue.
12. Push "character to write" on the stack.
13. Pull from right end, and push on left end till you push $.
14. Output everything on the stack to the tape of QA.
15. If the character after H is not a $. Do the following.
16. Push the next character on to the stack.
17. Remove the first character on right end, and dump it.
18. Rotate once, i.e. pull H from right end and push it in.
19. Push character to be written in here.
20. Rotate till you push in H again.
21. Output everything on our stack at this point.

So lets look at the flowing example of ($,k,H,b,c,d,e,#).
Now we want to write to "b", and move left.
For ex: lets say we want to write "o" , and move left.
Therefore, if we pop "b" then "H" and then "k".
We push in, "o" then "k" and then "H".
Therefore, we have ($,H,k,o,c,d,e,#).
Thus, we have simulated a left move too.

A key difference between QA and PDA, is that in a PDA, we have to push to top of stack, so we cannot cycle through the stack, but in a QA since , we pop from right end, and push into the left end, we can cycle through the stack to iterate through the inputs.

Since, we have simulated a read, write, left, and right move. It is evident that a QA can simulate a TM.
We have proved that a TM can be simulated on a QA, and that a QA can be simulated on a TM. Thus, we have proved that any language that is Turing recognizable is QA recognizable and vice versa.

$\square$